🧩 CATEGORY 7: Modules

> **Difficulty:** Intermediate → Advanced | **Topics:** 8 | **Terraform Interview Mastery Series**

Table of Contents

1. What Modules Are — Why They Exist, Anatomy
2. Module Sources — Local, Git, Terraform Registry
3. ⚠ Module Versioning Strategies
4. Passing Inputs In, Pulling Outputs Out
5. ⚠ Module Composition Patterns — Root, Child, Wrapper Modules
6. `moved` Block — Refactoring Without Destroy/Recreate
7. ⚠ `terraform import` — Importing Existing Infrastructure
8. Public vs Private Module Registry

Topic 47: What Modules Are — Why They Exist, Anatomy

🔵 What It Is (Simple Terms)

A **module** is a container for a group of Terraform resources that are used together. Every Terraform configuration is technically a module — the files in your working directory form the **root module**. When you call another directory or registry source, you're using a **child module**.

Think of modules like functions in programming — they encapsulate logic, accept inputs (variables), and return outputs. You call them once and reuse across environments, teams, and projects.

🔵 Why Modules Exist — The Problems They Solve

```
Without modules:
  ❌ Copy-paste the same VPC config for dev, staging, prod
     → 3 copies to maintain, 3 places to fix bugs
  ❌ 500-line main.tf files with VPC + IAM + RDS + EC2 all mixed together
     → Hard to understand, dangerous to change
  ❌ No standard patterns — every team reinvents the wheel
     → Inconsistent tagging, naming, security settings
  ❌ No way to enforce best practices across the organization

With modules:
  ✅ Define VPC logic once → call it three times for dev/staging/prod
  ✅ Separate VPC, IAM, RDS concerns into focused, testable units
  ✅ Platform team publishes approved modules → app teams consume
```

```
  ✅ Module enforces security defaults — consumers can't forget them
  ✅ Module is versioned — breaking changes are controlled
```

🔵 Every Config Is Already a Module

```
The working directory = root module
All .tf files in the directory are part of the root module

myproject/
├── main.tf          ← resource definitions
├── variables.tf     ← input variable declarations
├── outputs.tf       ← output value declarations
├── versions.tf      ← required_providers, required_version
├── locals.tf        ← local value definitions
└── data.tf          ← data source definitions

This IS a module. When you run terraform plan in this directory,
you're executing the root module.
```

🔵 Full Module Anatomy

Module Directory Structure

```
modules/
└── vpc/                          ← module directory
    ├── main.tf                   ← resource definitions (required)
    ├── variables.tf              ← input declarations (required for
interface)
    ├── outputs.tf                ← output declarations (required for
interface)
    ├── versions.tf               ← provider requirements (recommended)
    ├── locals.tf                 ← internal computed values (optional)
    ├── data.tf                   ← data sources (optional)
    └── README.md                 ← documentation (strongly recommended)
```

`variables.tf` — The Module's Input Interface

```
# modules/vpc/variables.tf

variable "vpc_cidr" {
  type        = string
  description = "CIDR block for the VPC"
  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
```

```
    error_message = "Must be a valid CIDR block."
  }
}

variable "environment" {
  type        = string
  description = "Environment name: dev, staging, prod"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}

variable "az_count" {
  type        = number
  description = "Number of availability zones to deploy subnets across"
  default     = 2
  validation {
    condition     = var.az_count >= 1 && var.az_count <= 3
    error_message = "az_count must be between 1 and 3."
  }
}

variable "enable_nat_gateway" {
  type        = bool
  description = "Create NAT gateways for private subnet internet access"
  default     = true
}

variable "tags" {
  type        = map(string)
  description = "Additional tags to apply to all resources"
  default     = {}
}
```

`main.tf` — The Module's Resources

```
# modules/vpc/main.tf

locals {
  name_prefix = "${var.environment}-vpc"
  common_tags = merge(var.tags, {
    Environment = var.environment
    ManagedBy   = "terraform"
    Module      = "vpc"
  })
}

data "aws_availability_zones" "available" {
  state = "available"
}
```

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = merge(local.common_tags, { Name = local.name_prefix })
}

resource "aws_subnet" "private" {
  count             = var.az_count
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone =
data.aws_availability_zones.available.names[count.index]

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-private-${count.index + 1}"
    Tier = "private"
  })
}

resource "aws_subnet" "public" {
  count                   = var.az_count
  vpc_id                  = aws_vpc.main.id
  cidr_block              = cidrsubnet(var.vpc_cidr, 8, count.index + 100)
  availability_zone       =
data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-public-${count.index + 1}"
    Tier = "public"
  })
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = merge(local.common_tags, { Name = "${local.name_prefix}-igw" })
}

resource "aws_eip" "nat" {
  count  = var.enable_nat_gateway ? var.az_count : 0
  domain = "vpc"
  tags   = merge(local.common_tags, { Name = "${local.name_prefix}-nat-
eip-${count.index + 1}" })
}

resource "aws_nat_gateway" "main" {
  count         = var.enable_nat_gateway ? var.az_count : 0
  allocation_id = aws_eip.nat[count.index].id
  subnet_id     = aws_subnet.public[count.index].id
  tags          = merge(local.common_tags, { Name = "${local.name_prefix}-
nat-${count.index + 1}" })
```

```
  depends_on    = [aws_internet_gateway.main]
}
```

`outputs.tf` — The Module's Public Interface

```
# modules/vpc/outputs.tf

output "vpc_id" {
  description = "ID of the created VPC"
  value       = aws_vpc.main.id
}

output "vpc_cidr" {
  description = "CIDR block of the VPC"
  value       = aws_vpc.main.cidr_block
}

output "private_subnet_ids" {
  description = "IDs of private subnets"
  value       = aws_subnet.private[*].id
}

output "public_subnet_ids" {
  description = "IDs of public subnets"
  value       = aws_subnet.public[*].id
}

output "nat_gateway_ids" {
  description = "IDs of NAT gateways (empty list if disabled)"
  value       = aws_nat_gateway.main[*].id
}
```

`versions.tf` — Provider Requirements

```
# modules/vpc/versions.tf

terraform {
  required_version = ">= 1.3.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0, < 6.0"    # permissive range — modules should NOT
pin
    }
  }
}
```

🔵 Calling the Module

```
# root module: main.tf

module "vpc" {
  source = "./modules/vpc"       # local path

  # Required inputs
  vpc_cidr    = "10.0.0.0/16"
  environment = var.environment

  # Optional inputs (module has defaults)
  az_count           = 3
  enable_nat_gateway = var.environment == "prod" ? true : false

  tags = {
    Project    = "myapp"
    CostCenter = "platform"
  }
}

# Accessing module outputs
resource "aws_eks_cluster" "main" {
  name = "my-cluster"

  vpc_config {
    subnet_ids = module.vpc.private_subnet_ids   # module.<name>.<output>
  }
}

output "vpc_id" {
  value = module.vpc.vpc_id    # re-export module output
}
```

🔵 Short Interview Answer

> "A module is a container for a group of related Terraform resources — every configuration is technically a module (the root module). Child modules encapsulate reusable infrastructure patterns: a VPC module, a database module, a Kubernetes cluster module. The module interface is defined by its `variables.tf` (inputs) and `outputs.tf` (public API). Modules solve three problems: eliminating copy-paste repetition, enforcing organizational standards (security, naming, tagging), and enabling platform teams to provide pre-approved infrastructure building blocks that app teams consume. Module versioning ensures breaking changes are controlled and gradual."

🔵 Gotchas & Edge Cases

- ⚠ **Module path changes state addresses** — `aws_vpc.main` becomes `module.vpc.aws_vpc.main`. Adding a module wrapper around existing resources requires `moved`

blocks.

- **Modules don't inherit provider aliases** — if root has `aws.us_east_1` alias, the module doesn't see it unless explicitly passed via `providers` map.
- **No module-level backend** — modules don't have their own state. State is always owned by the root module that calls the modules.
- **Deep module nesting is an anti-pattern** — module calling module calling module creates dependency chains that are hard to debug. Usually two levels (root + child) is sufficient.

Topic 48: Module Sources — Local, Git, Terraform Registry

🔵 What It Is (Simple Terms)

The `source` argument in a module block tells Terraform **where to find the module code**. Terraform supports several source types: local filesystem paths, Git repositories, Terraform Registry, S3 buckets, and more.

🔵 Local Path Source

```
# Relative path — most common for modules within the same repo
module "vpc" {
  source = "./modules/vpc"       # relative to the calling module
}

module "rds" {
  source = "./modules/rds"
}

module "shared_vpc" {
  source = "../shared/modules/vpc"  # parent directory
}

# Absolute paths work but are fragile (don't use in team repos)
module "vpc" {
  source = "/home/user/infra/modules/vpc"   # ❌ don't do this
}
```

When to use local paths:

- Monorepo: modules live in the same repo as the root config
- Development: building and testing a module before publishing
- Organization-specific modules that won't be shared externally

---

🔵 Git Source

```
# HTTPS
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc"
  #                                                                  ^^
  # Double slash separates repo URL from subdirectory path
}

# SSH
module "vpc" {
  source = "git::git@github.com:myorg/terraform-modules.git//modules/vpc"
}

# With version pinning via ref (CRITICAL for production)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v2.1.0"
  # ref can be: tag, branch, commit SHA
}

# Tag - most stable (immutable)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v2.1.0"
}

# Commit SHA - most precise (truly immutable)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=abc1234def5678"
}

# Branch - least stable (moves with new commits)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=main"
  # ⚠ Never use a branch in production - it's a moving target
}

# GitHub shorthand (Terraform resolves to HTTPS)
module "vpc" {
  source = "github.com/myorg/terraform-modules//modules/vpc?ref=v2.1.0"
}

# GitLab shorthand
module "vpc" {
  source = "gitlab.com/myorg/terraform-modules//modules/vpc?ref=v2.1.0"
}

# BitBucket shorthand
```

```
module "vpc" {
  source = "bitbucket.org/myorg/terraform-modules//modules/vpc?ref=v2.1.0"
}
```

🔵 Terraform Registry Source

```
# Public Registry — format: <namespace>/<module_name>/<provider>
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"             # version constraint required for
registry
}

module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "= 20.8.4"          # exact pin for production
}

module "rds" {
  source  = "terraform-aws-modules/rds/aws"
  version = ">= 6.0, < 7.0"
}

# Private Registry (Terraform Cloud / Terraform Enterprise)
# format: <hostname>/<namespace>/<module_name>/<provider>
module "vpc" {
  source  = "app.terraform.io/mycompany/vpc/aws"
  version = "2.1.0"
}
```

🔵 Other Source Types

```
# S3 bucket (private module storage)
module "vpc" {
  source = "s3::https://s3-eu-west-1.amazonaws.com/mycompany-
modules/vpc.zip"
}

# GCS bucket
module "vpc" {
  source = "gcs::https://www.googleapis.com/storage/v1/mycompany-
modules/vpc.zip"
}

# HTTP URL (generic archive)
module "vpc" {
  source = "https://example.com/modules/vpc.zip"
```

```
}

# Mercurial (hg)
module "vpc" {
  source = "hg::http://bitbucket.org/myorg/modules//vpc"
}
```

🔵 The `//` Double Slash Convention

```
# Double slash separates the repository URL from the subdirectory
# Use when your module is in a subdirectory of a repo

# Entire repo IS the module (no subdirectory needed)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-aws-vpc.git"
  # Module code is at the root of the repo
}

# Module is in a subdirectory of a larger repo
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc"
  #                                                                  ^^
subdirectory
}

# Multiple modules in one repo, each in its own directory
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v1.0"
}
module "rds" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/rds?ref=v1.0"
}
module "eks" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/eks?ref=v1.0"
}
```

🔵 `terraform init` and Module Sources

```
# terraform init downloads modules to .terraform/modules/
terraform init

# .terraform/modules/ structure:
.terraform/
```

```
└── modules/
    ├── modules.json          ← registry of all downloaded modules
    ├── vpc/                  ← local path module (symlink or copy)
    └── eks/                  ← downloaded module
        ├── main.tf
        ├── variables.tf
        └── outputs.tf

# Re-download/update modules
terraform init -upgrade       # downloads newer versions matching
constraints

# .terraform/modules/ should be in .gitignore
# modules are downloaded fresh on each terraform init
```

🔵 Short Interview Answer

> "Module sources tell Terraform where to find module code. Local paths (`./modules/vpc`) are used for modules in the same repo. Git sources (`git::https://...`) are used for shared modules across repos — always pin with `?ref=v2.1.0` using a tag, never a branch. Terraform Registry (`namespace/module/provider`) is the public registry with community and partner modules — requires a `version` constraint. Private registries use Terraform Cloud or Enterprise. The double-slash convention (`repo.git//subdirectory`) separates the repo URL from the module's subdirectory path within the repo. In production, always pin to immutable references — tags or commit SHAs — never floating references like branches."

Topic 49: ⚠ Module Versioning Strategies

🔵 What It Is (Simple Terms)

Module versioning controls **which version of a module your infrastructure uses** and **how upgrades are managed**. Poor versioning strategy leads to surprise breaking changes in production or difficulty upgrading over time.

> ⚠ Module versioning is an area where interviewers separate candidates who've managed modules in production from those who've only read docs.

🔵 Version Constraints for Registry Modules

```
# Exact pin — safest for production, hardest to upgrade
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "= 5.1.4"
}
```

```
# Pessimistic constraint — allows patches, not minor/major
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.1"      # allows 5.1.x, NOT 5.2.x or 6.x
}

module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"      # allows 5.x, NOT 6.x
}

# Range constraint — explicit bounds
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = ">= 5.0, < 6.0"    # equivalent to ~> 5.0
}

# Minimum version — dangerous, allows any major version upgrade
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = ">= 5.0"    # ❌ Could pick up v6.0 with breaking changes
}
```

🔵 Versioning Strategies for Internal/Git Modules

```
# Strategy 1: Semantic versioning with Git tags (recommended)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v2.1.0"
}

# Semantic versioning format: MAJOR.MINOR.PATCH
# MAJOR: breaking changes (variable renamed, resource restructured)
# MINOR: new features backward-compatible (new optional variable)
# PATCH: bug fixes backward-compatible

# Strategy 2: Commit SHA pinning (most immutable)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=a1b2c3d4e5f6789"
}
# Pros: truly immutable, tag can be reassigned (force push), SHA can't
# Cons: unreadable, hard to understand what version you're on

# Strategy 3: Branch pinning (DANGEROUS — do not use in production)
module "vpc" {
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=main"
}
```

```
# This changes every time a commit is pushed to main
# Next terraform init gets a different module version
# Silent breaking changes
```

🔵 Module Author Versioning Rules

```
# As the module author, follow these rules:

# In reusable modules — permissive constraints
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0, < 6.0"   # ✅ Permissive — accepts many provider
versions
    }
  }
}

# In root modules — exact or tight constraints
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.1"           # ✅ Pinned for root module
reproducibility
    }
  }
}

# Why the difference?
# Reusable module: consumed by many callers with different provider
versions
# → Overly restrictive = incompatibility with callers
# Root module: pinned for reproducible builds
# → Loosely pinned = different CI runs get different provider versions
```

🔵 Managing Module Upgrades Safely

```
# Step 1: Review the changelog
# Check GitHub releases page or CHANGELOG.md
# Look for: breaking changes, deprecated variables, renamed outputs

# Step 2: Upgrade in development first
# dev/terraform.tfvars:
#   module version = "v3.0.0"  ← update here first
```

```
# Step 3: Run plan on dev
terraform plan
# Look for: unexpected destroys, resource recreation, plan errors

# Step 4: Check for moved blocks in the new module
# Good module authors include moved blocks for renamed resources
# Bad module authors just rename and let you figure it out

# Step 5: Migrate state if needed
# If module restructured resources, may need terraform state mv

# Step 6: Test thoroughly in dev/staging before prod

# Step 7: Upgrade prod with same version
# After verifying dev/staging works correctly
```

🔵 Lock File and Module Versions

```
# .terraform.lock.hcl locks PROVIDER versions, NOT module versions
# Module versions are controlled by the source + version in your config
# There is no automatic lock file for modules

# This means:
# - You must pin module versions in your config
# - Without pinning, terraform init -upgrade could get a different version
# - Always use explicit ?ref= for Git modules and version= for Registry
modules
```

🔵 The Upgrade Anti-Pattern

```
# ❌ The dangerous upgrade anti-pattern:
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = ">= 3.0"   # accepts 3.x, 4.x, 5.x, 6.x...
}

# What happens:
# - Team A: terraform init → gets v4.5.0
# - Team B: terraform init → gets v5.0.0 (breaking change!)
# - CI/CD: terraform init → gets latest = v5.1.0
# - Everyone is running a different module version
# - Plans differ between team members and CI
# ← This is non-reproducible infrastructure

# ✅ Correct pattern:
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
```

```
  version = "= 5.1.4"   # exact same version for everyone
}
```

🔵 Short Interview Answer

> "Module versioning strategy depends on the source type. For Terraform Registry modules, always use an explicit `version` constraint — exact pin (`= 5.1.4`) for maximum stability in production, or pessimistic constraint (`~> 5.0`) that allows patch updates. For Git modules, always use `?ref=v2.1.0` with a semantic version tag — never a branch, which is a moving target. As a module author, use permissive version constraints for providers (`>= 4.0, < 6.0`) so callers with different provider versions can consume your module. For root modules, use tight constraints for reproducibility. The upgrade process should always go dev → staging → prod, with plan review at each stage and checking the changelog for breaking changes."

Topic 50: Passing Inputs In, Pulling Outputs Out

🔵 What It Is (Simple Terms)

The **variable/output contract** is how information flows between modules — variables flow in (from caller to module), outputs flow out (from module to caller). Understanding this contract deeply is essential for designing composable, reusable modules.

🔵 Passing Inputs to Modules

```
module "vpc" {
  source = "./modules/vpc"

  # ── Required inputs (no default in module) ──────────────────────────
  vpc_cidr    = "10.0.0.0/16"
  environment = var.environment          # pass variable through

  # ── Optional inputs (module has defaults, you're overriding) ─────────
  az_count           = 3
  enable_nat_gateway = var.environment == "prod"

  # ── Computed inputs from resources ───────────────────────────────────
  kms_key_arn = aws_kms_key.main.arn     # resource attribute

  # ── Inputs from another module's outputs ─────────────────────────────
  security_group_ids = module.security.app_sg_ids   # from sibling module

  # ── Complex object inputs ────────────────────────────────────────────
  tags = merge(local.common_tags, { Component = "networking" })
```

```
  # ── Conditional inputs ──────────────────────────────────────────────
  db_instance_class = var.environment == "prod" ? "db.r5.large" :
"db.t3.medium"
}
```

🔵 Module Output Access Patterns

```
# Basic output access: module.<module_name>.<output_name>
resource "aws_instance" "app" {
  subnet_id = module.vpc.private_subnet_ids[0]
}

# Output from module used as input to another module
module "rds" {
  source = "./modules/rds"

  # Chaining module outputs
  vpc_id             = module.vpc.vpc_id
  subnet_ids         = module.vpc.private_subnet_ids
  security_group_ids = [module.security.rds_sg_id]
}

# Re-exporting module outputs at root level
output "vpc_id" {
  description = "The VPC ID"
  value       = module.vpc.vpc_id
}

output "database_endpoint" {
  description = "RDS connection endpoint"
  value       = module.rds.endpoint
  sensitive   = true
}

# Aggregating outputs from multiple modules
output "all_subnet_ids" {
  value = concat(
    module.vpc_primary.private_subnet_ids,
    module.vpc_secondary.private_subnet_ids
  )
}
```

🔵 What Happens With Missing Required Inputs

```
# If you call a module without a required variable (no default):
module "vpc" {
  source = "./modules/vpc"
```

```
  # missing: vpc_cidr (required)
  environment = "prod"
}

# terraform plan error:
# Error: Missing required argument
# The argument "vpc_cidr" is required, but no definition was found.
# on main.tf line 1, in module "vpc":
# 1: module "vpc" {
```

🔵 Optional Module Arguments with `optional()` (Terraform 1.3+)

```
# Module variable definition with optional object attributes
variable "database_config" {
  type = object({
    instance_class     = string
    allocated_storage  = optional(number, 100)    # optional, default 100
    multi_az           = optional(bool, false)    # optional, default
false
    backup_retention   = optional(number, 7)      # optional, default 7
    deletion_protection = optional(bool, true)    # optional, default true
  })
}

# Caller only needs to provide required attributes
module "rds" {
  source = "./modules/rds"

  database_config = {
    instance_class = "db.t3.medium"
    # all other attributes use defaults from optional()
  }
}
```

🔵 Module Output Sensitivity

```
# If a module output is sensitive, the root module must handle it

# modules/rds/outputs.tf
output "master_password" {
  value     = random_password.db.result
  sensitive = true
}

# Root module — accessing sensitive output
output "db_password" {
  value     = module.rds.master_password
```

```
  sensitive = true   # ← MUST mark sensitive here too or Terraform errors
}

# Using sensitive output in a resource (no special handling needed)
resource "aws_secretsmanager_secret_version" "db" {
  secret_id     = aws_secretsmanager_secret.db.id
  secret_string = module.rds.master_password   # works fine — sensitivity
propagates
}
```

🔵 Short Interview Answer

> "Inputs flow into modules via the `module` block arguments — each argument corresponds to a variable declared in the module's `variables.tf`. Outputs flow out via the module's `outputs.tf` and are accessed as `module.<name>.<output>` in the calling config. Required variables (no default) must be provided or Terraform errors at plan time. Sensitive outputs from child modules must be marked `sensitive = true` when re-exported from root, otherwise Terraform 0.15+ errors. Outputs can chain between modules — `module.rds.subnet_ids = module.vpc.private_subnet_ids` — enabling composable infrastructure patterns where modules build on each other's outputs."

Topic 51: ⚠ Module Composition Patterns — Root, Child, Wrapper Modules

---

🔵 What It Is (Simple Terms)

Module composition is how you **architect multiple modules together** to build complete infrastructure. There are distinct patterns for how modules relate to each other — understanding these patterns separates engineers who write good module architecture from those who create tangled, unmaintainable configs.

🔵 The Three Module Roles

```
ROOT MODULE:
  The entry point — where terraform apply is run
  Calls child modules, owns the state file
  Environment-specific (one root per environment or workspace)
  Should be thin — mostly module calls and wiring
  Does NOT contain raw resources (ideally)

CHILD MODULE:
  A reusable unit of infrastructure
  Called by root module (or wrapper module)
  Accepts inputs, produces outputs
```

```
  Should be focused on one concern (VPC, RDS, EKS)
  Should NOT know about the environment it's deployed into

WRAPPER MODULE:
  Wraps another module (often a third-party one)
  Adds organizational defaults (tags, naming, security settings)
  Narrows the interface — exposes only what callers should configure
  Hides complexity of the underlying module
```

🔵 Pattern 1: Flat Module Composition (Most Common)

```
root/
├── main.tf           ← calls all modules
├── variables.tf
├── outputs.tf
└── modules/
    ├── vpc/
    ├── security/
    ├── rds/
    └── eks/
```

```
# root/main.tf — flat composition

module "vpc" {
  source = "./modules/vpc"
  vpc_cidr    = var.vpc_cidr
  environment = var.environment
}

module "security" {
  source = "./modules/security"
  vpc_id      = module.vpc.vpc_id
  environment = var.environment
}

module "rds" {
  source = "./modules/rds"
  vpc_id             = module.vpc.vpc_id
  subnet_ids         = module.vpc.private_subnet_ids
  security_group_ids = [module.security.rds_sg_id]
  environment        = var.environment
}

module "eks" {
  source = "./modules/eks"
  vpc_id             = module.vpc.vpc_id
  subnet_ids         = module.vpc.private_subnet_ids
  security_group_ids = [module.security.eks_sg_id]
```

```
  environment        = var.environment
}
```

🔵 Pattern 2: Wrapper Module (Corporate Standard Enforcement)

```
# Problem: terraform-aws-modules/vpc/aws has 200+ variables
# Your org only needs 10 of them, with specific defaults for others
# Every team shouldn't configure security settings independently

# Solution: Wrapper module that narrows the interface

# modules/vpc-wrapper/main.tf
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "= 5.1.4"

  name = "${var.environment}-vpc"
  cidr = var.vpc_cidr

  azs             = slice(data.aws_availability_zones.available.names, 0,
var.az_count)
  private_subnets = [for i in range(var.az_count) :
cidrsubnet(var.vpc_cidr, 8, i)]
  public_subnets  = [for i in range(var.az_count) :
cidrsubnet(var.vpc_cidr, 8, i + 100)]

  # ── ENFORCED organizational defaults (not configurable by callers) ──
  enable_nat_gateway   = true          # ALWAYS enable NAT
  single_nat_gateway   = var.environment != "prod"  # prod gets HA NAT
  enable_dns_hostnames = true          # ALWAYS enable DNS
  enable_dns_support   = true

  # ── ENFORCED security settings ──────────────────────────────────────
  enable_flow_log                      = true    # ALWAYS enable flow logs
  flow_log_destination_type            = "cloud-watch-logs"
  create_flow_log_cloudwatch_log_group = true
  create_flow_log_cloudwatch_iam_role  = true

  # ── Standard tagging ────────────────────────────────────────────────
  tags = merge(var.tags, {
    Environment = var.environment
    ManagedBy   = "terraform"
    Module      = "vpc-wrapper"
  })
}

# modules/vpc-wrapper/variables.tf — NARROW interface
variable "vpc_cidr"    { type = string }
variable "environment" { type = string }
variable "az_count"    { type = number; default = 2 }
```

```
variable "tags"        { type = map(string); default = {} }
# ← Only 4 variables instead of 200+
# Org standards are baked in, not configurable

# modules/vpc-wrapper/outputs.tf — pass through relevant outputs
output "vpc_id"             { value = module.vpc.vpc_id }
output "private_subnet_ids" { value = module.vpc.private_subnets }
output "public_subnet_ids"  { value = module.vpc.public_subnets }
```

🔵 Pattern 3: Service Module (Multiple Modules Combined)

```
# A "service module" bundles everything needed for one application

# modules/web-service/main.tf
# This module creates everything a web service needs

module "security_groups" {
  source      = "../security-groups"
  vpc_id      = var.vpc_id
  service_name = var.service_name
}

resource "aws_ecs_service" "main" {
  name            = var.service_name
  cluster         = var.ecs_cluster_id
  task_definition = aws_ecs_task_definition.main.arn
  desired_count   = var.desired_count

  network_configuration {
    subnets         = var.subnet_ids
    security_groups = [module.security_groups.service_sg_id]
  }
}

resource "aws_lb_target_group" "main" {
  name     = "${var.service_name}-tg"
  port     = var.container_port
  protocol = "HTTP"
  vpc_id   = var.vpc_id
}

resource "aws_lb_listener_rule" "main" {
  listener_arn = var.lb_listener_arn
  condition {
    host_header { values = ["${var.service_name}.${var.domain}"] }
  }
  action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.main.arn
  }
```

```
}

resource "aws_cloudwatch_log_group" "main" {
  name              = "/ecs/${var.service_name}"
  retention_in_days = 30
}
```

🔵 Pattern 4: Multi-Environment Using Modules

```
environments/
├── dev/
│   ├── main.tf        ← calls modules with dev config
│   ├── backend.tf
│   └── terraform.tfvars
├── staging/
│   ├── main.tf        ← calls same modules with staging config
│   ├── backend.tf
│   └── terraform.tfvars
└── prod/
    ├── main.tf        ← calls same modules with prod config
    ├── backend.tf
    └── terraform.tfvars
modules/
├── vpc/
├── rds/
└── eks/
```

```
# environments/prod/main.tf
module "vpc" {
  source = "../../modules/vpc"
  # prod-specific config
  vpc_cidr           = "10.0.0.0/16"
  environment        = "prod"
  az_count           = 3
  enable_nat_gateway = true
}

# environments/dev/main.tf
module "vpc" {
  source = "../../modules/vpc"
  # dev-specific config
  vpc_cidr           = "10.1.0.0/16"
  environment        = "dev"
  az_count           = 2
  enable_nat_gateway = false   # save cost in dev
}
```

🔵 Module Depth — How Deep Is Too Deep?

```
Acceptable: 2 levels (most common)
  root → vpc, rds, eks, security

Acceptable: 3 levels (for complex cases)
  root → platform-module → vpc, rds, eks

Avoid: 4+ levels
  root → platform → app-stack → service → vpc

Why deep nesting is bad:
  - Long state addresses:
module.platform.module.app.module.vpc.aws_vpc.main
  - Difficult to debug which module is creating what
  - Hard to test modules independently
  - Output chains: root needs to pass outputs up through every layer
  - Slow plan — Terraform must process every layer
```

🔵 Short Interview Answer

> "Module composition has three key roles. Root modules are the entry points per environment — thin orchestrators that call child modules and wire their outputs together. Child modules are focused reusable units — VPC module, RDS module, EKS module. Wrapper modules wrap third-party or complex modules to enforce organizational standards — they narrow the interface to only what callers should configure and bake in security defaults so teams can't accidentally disable them. The most important design principle: child modules should be self-contained and not know about the environment. Root modules apply environment-specific config by passing appropriate variable values. Keep depth to two levels — root and children — to avoid complex dependency chains."

Topic 52: `moved` Block — Refactoring Without Destroy/Recreate

🔵 What It Is (Simple Terms)

The `moved` block (Terraform 1.1+) tells Terraform that a resource has been **renamed or moved** in the configuration, without destroying and recreating it. Terraform updates only the state file to reflect the new address.

🔵 Why It Exists — The Rename Problem

```
# Before rename:
resource "aws_instance" "web" { ... }
# State: aws_instance.web → i-abc123

# After rename (without moved block):
resource "aws_instance" "web_server" { ... }
# Terraform sees: new resource "web_server" to create
#                 old resource "web" to destroy
# Plan: 1 to add, 0 to change, 1 to destroy  ← DESTROYS THE INSTANCE!

# After rename (WITH moved block):
resource "aws_instance" "web_server" { ... }
moved {
  from = aws_instance.web
  to   = aws_instance.web_server
}
# Plan: Terraform will move aws_instance.web to aws_instance.web_server
# Plan: 0 to add, 0 to change, 0 to destroy  ← PERFECT
```

🔵 Full `moved` Block Syntax and Use Cases

Renaming a Resource

```
# Rename resource within the same module
moved {
  from = aws_instance.web
  to   = aws_instance.web_server
}

# Rename a module
moved {
  from = module.database
  to   = module.rds
}
```

Moving into a Module

```
# Moving a resource from root into a child module
moved {
  from = aws_vpc.main
  to   = module.networking.aws_vpc.main
}

moved {
  from = aws_subnet.private[0]
  to   = module.networking.aws_subnet.private[0]
}
```

```
moved {
  from = aws_subnet.private[1]
  to   = module.networking.aws_subnet.private[1]
}
```

Moving Between Modules

```
# Restructuring — moving resource from one child module to another
moved {
  from = module.app_v1.aws_instance.web
  to   = module.app_v2.aws_instance.web
}
```

`count` to `for_each` Migration

```
# Migrating from count to for_each (see Category 11 Topic 98)
moved {
  from = aws_instance.web[0]
  to   = aws_instance.web["web-a"]
}

moved {
  from = aws_instance.web[1]
  to   = aws_instance.web["web-b"]
}
```

For_each Key Rename

```
# Renaming a for_each key
moved {
  from = aws_instance.web["old-key"]
  to   = aws_instance.web["new-key"]
}
```

🔵 How `moved` Works

```
1. Terraform reads the moved block: from = A, to = B
2. Terraform looks up A's state entry
3. Terraform renames the state entry from A to B
4. Terraform checks: does resource B exist in config? Yes → great
5. Plan shows: "Terraform will move resource A to B"
```

```
6. Apply: updates state only — NO API calls to cloud provider
7. Real infrastructure is UNTOUCHED
```

🔵 Multiple `moved` Blocks and Order

```
# Multiple moves can be in the same file or separate files
# Order doesn't matter — Terraform processes them all

moved {
  from = aws_vpc.main
  to   = module.networking.aws_vpc.main
}

moved {
  from = aws_subnet.private[0]
  to   = module.networking.aws_subnet.private[0]
}

moved {
  from = aws_subnet.private[1]
  to   = module.networking.aws_subnet.private[1]
}

moved {
  from = aws_internet_gateway.main
  to   = module.networking.aws_internet_gateway.main
}

# Terraform processes all of these atomically in one plan/apply
```

🔵 Chained `moved` Blocks

```
# You can chain moves — A was renamed to B, B is now renamed to C
moved {
  from = aws_instance.web
  to   = aws_instance.web_server
}

moved {
  from = aws_instance.web_server
  to   = module.app.aws_instance.web_server
}

# Terraform resolves the chain: aws_instance.web →
module.app.aws_instance.web_server
```

🔵 Where to Put `moved` Blocks

```
# Option 1: In a dedicated moved.tf file (keeps things clean)
# moved.tf
moved {
  from = aws_instance.web
  to   = aws_instance.web_server
}

# Option 2: In the same file as the resource
# main.tf
resource "aws_instance" "web_server" { ... }

moved {
  from = aws_instance.web
  to   = aws_instance.web_server
}

# Option 3: In a refactoring PR — all moves for one refactoring together
# refactor-networking.tf
moved { from = aws_vpc.main; to = module.networking.aws_vpc.main }
moved { from = aws_subnet.private[0]; to =
module.networking.aws_subnet.private[0] }
```

🔵 When to Remove `moved` Blocks

```
Keep moved blocks UNTIL all environments have applied the change:
  dev:     applied ✅ → can remove
  staging: applied ✅ → can remove
  prod:    applied ✅ → now safe to remove

Remove moved blocks in a separate PR after all environments are updated
Add a comment in the removal PR explaining what was moved and when

Why not keep them forever?
  → Accumulate over time and clutter the codebase
  → Plan output becomes noisy with "will move N resources" for already-
moved resources

Why you CAN'T remove them early:
  → If an environment hasn't applied the move yet, removing the block
    means that environment's next plan shows destroy + create again
    (back to the original problem)
```

🔵 `moved` vs `terraform state mv`

```
terraform state mv (imperative):
  ✅ Works in all Terraform versions
  ✅ Immediate — changes state right now
  ✅ Useful for one-time emergency renames
  ❌ Not codified — not visible in config history
  ❌ Must be run manually on every environment
  ❌ Easy to forget to run on prod after running on dev

moved block (declarative):
  ✅ Codified — visible in config, committed to Git
  ✅ Automatic — anyone who runs plan/apply gets the move
  ✅ Self-documenting — clear intent in the codebase
  ✅ Applies to all environments automatically
  ✅ Part of the normal PR/review/apply workflow
  ❌ Requires Terraform 1.1+

Preference: moved block for all planned refactoring
            terraform state mv for emergency one-off operations
```

🔵 Short Interview Answer

> "The `moved` block tells Terraform that a resource has been renamed or relocated in configuration — it updates only the state file without touching real infrastructure. It's the declarative, codified alternative to `terraform state mv`. Define the old address in `from` and the new address in `to`, and Terraform's plan shows 'will move resource A to B' with zero creates or destroys. Use cases: renaming resources, moving into or between modules, migrating from `count` to `for_each`. Keep `moved` blocks until all environments have applied the change, then remove them. The key advantage over `state mv` is it's codified in config — committed to Git, part of the normal PR workflow, and automatically applied to every environment without manual CLI intervention."

Topic 53: ⚠ `terraform import` — Importing Existing Infrastructure

🔵 What It Is (Simple Terms)

`terraform import` brings **existing cloud resources under Terraform management** — adding them to the state file without recreating them. Used when infrastructure was created manually, by another tool, or by another Terraform config.

> ⚠ Import is one of the most interview-tested operations because it requires understanding both Terraform and the provider's import syntax.

🔵 Why Import Is Needed

```
Scenario 1: Legacy infrastructure
  Pre-Terraform infrastructure exists in AWS
  You want Terraform to manage it going forward
  Solution: Import + write matching config

Scenario 2: Another tool managed it
  CloudFormation stack created an RDS instance
  You're migrating to Terraform
  Solution: Import the RDS instance

Scenario 3: Config/state drift
  State was lost, resources still exist
  Solution: Import resources back into state

Scenario 4: Terraform state rm was used
  Resource was removed from state but not destroyed
  Now you want to re-manage it
  Solution: Import it back
```

🔵 Classic `terraform import` CLI (Pre-1.5)

```
# Syntax: terraform import <resource_address> <provider_specific_id>

# Import an EC2 instance
terraform import aws_instance.web i-0abc123def456789

# Import an S3 bucket
terraform import aws_s3_bucket.data mycompany-prod-data

# Import an IAM role
terraform import aws_iam_role.lambda_role prod-lambda-execution-role

# Import an RDS instance
terraform import aws_db_instance.production prod-database

# Import a for_each resource (note quotes)
terraform import 'aws_instance.web["server-a"]' i-0abc123def456789

# Import a module resource
terraform import module.networking.aws_vpc.main vpc-0abc123def456789

# Import a count-indexed resource
terraform import 'aws_instance.workers[0]' i-0abc123def456789
```

Where to find the import ID:

```
Each provider documents the import ID format in their resource docs.
Common patterns:

aws_instance              → instance ID (i-xxxxx)
aws_s3_bucket             → bucket name
aws_iam_role              → role name
aws_vpc                   → VPC ID (vpc-xxxxx)
aws_db_instance           → DB identifier (not ARN)
aws_security_group        → security group ID (sg-xxxxx)
aws_route53_record        → zone_id/name/type (complex)
aws_lb                    → ARN
aws_iam_policy_attachment → role/user/group + policy ARN (complex)
```

🔵 The Import Workflow (Classic)

```
# Step 1: Write the resource config in your .tf files
# The config must match the imported resource's attributes
resource "aws_s3_bucket" "data" {
  bucket = "mycompany-prod-data"
  tags = {
    Environment = "prod"
    ManagedBy   = "terraform"
  }
}

# Step 2: Run terraform import
terraform import aws_s3_bucket.data mycompany-prod-data
# Output: aws_s3_bucket.data: Importing from ID "mycompany-prod-data"...
# Output: aws_s3_bucket.data: Import prepared!
# Output: aws_s3_bucket.data: Refreshing state... [id=mycompany-prod-data]
# Output: Import successful!

# Step 3: Run terraform plan
terraform plan
# GOAL: plan shows 0 changes (your config matches reality perfectly)
# REALITY: plan almost always shows changes on first import
# These are the differences between what you wrote and actual resource
config

# Step 4: Update config to match reality
# Read terraform state show aws_s3_bucket.data to see all attributes
terraform state show aws_s3_bucket.data
# Update your .tf file to match the actual attribute values

# Step 5: Repeat plan until 0 changes
terraform plan   # should eventually show: No changes.

# Step 6: Commit
```

🔵 Native Import Block (Terraform 1.5+) — The Better Way

```
# Terraform 1.5+ — import blocks in config (declarative)

import {
  to = aws_instance.web
  id = "i-0abc123def456789"
}

resource "aws_instance" "web" {
  # Minimal config — terraform plan will show what attributes exist
  # Then fill in the config to match
}
```

```
# Terraform 1.5+: generate config automatically!
terraform plan -generate-config-out=generated.tf
# Terraform calls the provider's read API
# Generates a resource block with ALL actual attribute values
# You get a starting point that already matches reality

# Review generated.tf, clean it up, move to main.tf
# Run terraform plan — should be 0 changes immediately
```

🔵 `generate-config-out` — The Game Changer

```
# Terraform 1.5+ with -generate-config-out
terraform plan -generate-config-out=generated_config.tf

# generated_config.tf (auto-generated):
resource "aws_instance" "web" {
  ami                          = "ami-0c55b159cbfafe1f0"
  associate_public_ip_address = true
  availability_zone            = "us-east-1a"
  cpu_core_count               = 2
  cpu_threads_per_core         = 2
  disable_api_stop             = false
  disable_api_termination      = false
  ebs_optimized                = false
  get_password_data            = false
  hibernation                  = false
  id                           = "i-0abc123def456789"
  instance_initiated_shutdown_behavior = "stop"
  instance_type                = "t3.medium"
  key_name                     = "prod-key"
  monitoring                   = false
```

```
    placement_partition_number  = 0
    private_ip                  = "10.0.1.45"
    secondary_private_ips       = []
    security_groups             = []
    source_dest_check           = true
    subnet_id                   = "subnet-0abc123"
    tags = {
      Environment = "prod"
      Name        = "prod-web"
    }
    tenancy                     = "default"
    vpc_security_group_ids      = ["sg-0abc123"]
    # ... all attributes
  }

  # This is 90% of the work done automatically!
  # Review, remove read-only attributes (id, arn), clean up defaults
  # Run terraform plan → should show 0 changes
```

🔵 Importing Complex Resources

```
# Resources with complex import IDs — check provider docs

# Route53 record: zone_id/name/type
terraform import aws_route53_record.www Z1D633PJN98FT9/_www.example.com/A

# IAM policy attachment: role/user/group + policy ARN
terraform import aws_iam_role_policy_attachment.example \
  "prod-lambda-role/arn:aws:iam::123456789012:policy/MyPolicy"

# Security group rule: sg_id_type_protocol_from_port_to_port_source
terraform import aws_security_group_rule.example \
  "sg-0abc123_ingress_tcp_80_80_0.0.0.0/0"

# VPC peering
terraform import aws_vpc_peering_connection.example pcx-0abc123

# ECS service: cluster_name/service_name
terraform import aws_ecs_service.example "prod-cluster/web-service"
```

🔵 Bulk Import Strategy

```
# For importing many resources (e.g., migrating from CloudFormation):

# Step 1: List all resources in CloudFormation stack
aws cloudformation describe-stack-resources \
  --stack-name my-stack \
```

```
  --query 'StackResources[*].[ResourceType,PhysicalResourceId]'

# Step 2: Write import blocks for all resources
# import.tf:
import { to = aws_instance.web;    id = "i-0abc123" }
import { to = aws_s3_bucket.data;  id = "mycompany-data" }
import { to = aws_iam_role.lambda; id = "my-lambda-role" }

# Step 3: Generate config for all
terraform plan -generate-config-out=imported.tf

# Step 4: Review, clean up generated config

# Step 5: Plan until 0 changes for all resources

# Step 6: Remove import blocks (they're one-time use)
```

🔵 Short Interview Answer

> "`terraform import` brings existing cloud resources under Terraform management by writing their state entry without recreating them. The classic CLI syntax is `terraform import <address> <provider_id>` where the ID format varies by resource type. The workflow: write a matching resource block, import, run `terraform state show` to see all actual attributes, update the config to match, and repeat until `terraform plan` shows zero changes. Terraform 1.5+ introduced native import blocks in config and `-generate-config-out` which auto-generates the resource configuration by reading the actual resource — this eliminates most of the manual config-writing work. The most common mistake is writing config first and hoping it matches — always check `terraform state show` after importing to see what the provider actually read."

🔵 Common Interview Questions

Q: Can you import a resource that already exists in another state file?

> "Yes — but you need to remove it from the source state first with `terraform state rm`, then import into the new state. If you don't, both state files believe they manage the resource. This creates a conflict — the first apply from either config could try to modify or destroy the resource based on its own state. The safe sequence: `state rm` from source, `import` into destination, update both configs, verify both plans show zero changes."

Q: What happens if you import a resource but the config doesn't match perfectly?

> "`terraform plan` will show the differences as changes to apply — Terraform will attempt to update the resource to match config. This is often fine for minor differences. But for attributes that force replacement (like an EC2 AMI or security group description), plan will show a destroy + create. You must decide: update the config to match the real value, or let Terraform update the resource. The goal of import is a clean plan with zero changes — meaning config perfectly describes the current state of the resource."

Topic 54: Public vs Private Module Registry

🔵 What It Is (Simple Terms)

The **Terraform Registry** (registry.terraform.io) is the public repository for sharing Terraform modules. A **private registry** (Terraform Cloud/Enterprise) provides the same functionality but restricted to your organization — for internal modules you don't want to make public.

🔵 Public Terraform Registry

```
registry.terraform.io

Structure:
  <NAMESPACE>/<MODULE>/<PROVIDER>
  terraform-aws-modules/vpc/aws
  terraform-aws-modules/eks/aws
  hashicorp/consul/aws

Registry tiers:
  Verified (✅ badge): maintained by HashiCorp or verified partner
  Community:           any GitHub user can publish

Publishing requirements:
  - GitHub repository named: terraform-<PROVIDER>-<NAME>
  - Must have: main.tf, variables.tf, outputs.tf, README.md
  - Tagged with semantic versions: v1.0.0, v1.1.0, v2.0.0
  - Must pass automated registry checks

Access:
  Public — anyone can use without authentication
  Source syntax: "namespace/module/provider"
  Version required: version = "~> 5.0"
```

🔵 Evaluating Public Modules Before Use

```
Quality checklist before adopting a public module:

✅ Published by verified partner or well-known organization
   (terraform-aws-modules is maintained by Anton Babenko — widely trusted)

✅ Active maintenance
   Last commit: within 6 months
   Open issues: being triaged and closed
   Pull requests: being reviewed
```

```
✅ Version history shows semantic versioning discipline
   Not: v0.0.1 followed by v1.0.0 with no minor versions in between

✅ Comprehensive documentation
   README.md with examples, variable descriptions, output descriptions

✅ Validation rules on variables
   Not: variable "environment" { type = string } with no validation

✅ Known usage
   Terraform Registry shows download count
   GitHub stars, forks — community adoption

✅ License compatible with your use
   Apache 2.0, MIT — permissive and business-friendly

Red flags:
  ❌ Last commit 2+ years ago
  ❌ No version tags — only branch-based
  ❌ Zero tests (no examples/, no .github/workflows/)
  ❌ Single contributor with no community
  ❌ GPL license (copyleft — check with legal)
```

🔵 Private Module Registry — Terraform Cloud/Enterprise

```
# Private registry module source format:
# <HOSTNAME>/<NAMESPACE>/<MODULE>/<PROVIDER>

module "vpc" {
  source  = "app.terraform.io/mycompany/vpc/aws"
  version = "2.1.0"
}

# Terraform Cloud self-hosted:
module "vpc" {
  source  = "tfe.mycompany.com/mycompany/vpc/aws"
  version = "2.1.0"
}
```

Private Registry Features:

```
✅ Same UX as public registry — browse, version, document
✅ Access control — only organization members can see/use modules
✅ Module versions published from Git tags automatically
✅ Automatic documentation generation from README.md and variables
✅ Usage tracking — see which workspaces use which module version
```

```
✅ No-code modules (Terraform Cloud) — deploy via UI without writing HCL
✅ Policy integration — Sentinel policies can enforce module usage
```

🔵 Hosting Internal Modules — Strategy Options

```
Option 1: Terraform Cloud/Enterprise Private Registry
  ✅ Best UX — same experience as public registry
  ✅ Version browsing, documentation
  ✅ SSO/RBAC integration
  ❌ Requires TFC/TFE subscription

Option 2: Git Repository + Tags
  ✅ Free — works with any Git provider
  ✅ Simple — no additional tooling
  source = "git::https://github.com/myorg/terraform-
modules.git//modules/vpc?ref=v2.1.0"
  ❌ No registry UI — harder to discover available modules
  ❌ Manual tag management

Option 3: S3/GCS Bucket
  ✅ Works without Git — good for highly controlled environments
  ✅ Works in air-gapped environments
  source = "s3::https://s3.amazonaws.com/mycompany-modules/vpc-v2.1.0.zip"
  ❌ No versioning UI — must manage object keys manually
  ❌ No documentation browsing

Option 4: Internal Nexus/Artifactory Module Registry
  ✅ Integrates with existing artifact management
  ✅ Works in enterprise environments
  ❌ Additional setup and maintenance

Best practice recommendation:
  Small teams: Git + tags (simple, free)
  Medium teams: Terraform Cloud private registry (best UX)
  Enterprise: TFE private registry + Sentinel for module enforcement
```

🔵 Module Registry Governance

```
# In large organizations, you want to control WHICH modules teams can use
# Sentinel policy: enforce all modules come from private registry

# Sentinel policy (Terraform Cloud)
import "tfplan/v2" as tfplan

# Check all module sources
all_modules_from_private_registry = rule {
  all tfplan.module_calls as _, calls {
```

```
    all calls as _, call {
      call.source matches "^app\\.terraform\\.io/mycompany/.*"
    }
  }
}

main = rule {
  all_modules_from_private_registry
}

# This policy ENFORCES that teams can only use modules from
# the private registry — preventing use of untested public modules
# or random Git repos
```

🔵 Short Interview Answer

> "The public Terraform Registry at registry.terraform.io hosts community and partner modules — the `terraform-aws-modules` collection is the most widely used. Source format is `namespace/module/provider` with a required `version` constraint. Before adopting a public module, evaluate: is it actively maintained, does it have a verified badge, does it follow semantic versioning? Private registries — Terraform Cloud or Enterprise — provide the same UX but restricted to your organization. Internal modules are published from Git tags automatically. The advantage over raw Git sources: browsable UI, documentation, version history, and access control. For organizations that need to govern which modules teams can use, Sentinel policies can enforce that all modules come from the private registry."

📊 Category 7 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
47. Module anatomy	Variables = interface in, outputs = interface out, versions.tf permissive	⭐⭐⭐⭐⭐
48. Module sources	Local, Git (tag not branch!), Registry, double-slash for subdirs	⭐⭐⭐⭐
49. Versioning ⚠	Exact pin in root, permissive in reusable, never branch in prod	⭐⭐⭐⭐⭐
50. Inputs/outputs	module.., sensitive propagation, optional() for objects	⭐⭐⭐⭐
51. Composition ⚠	Root=orchestrator, child=focused unit, wrapper=standard enforcer	⭐⭐⭐⭐⭐
52. `moved` block	Declarative rename/move — zero destroy, codified in Git	⭐⭐⭐⭐⭐

Topic	One-Line Summary	Interview Weight
53. `terraform import` ⚠	Import ID format, generate-config-out, clean plan = success	⭐⭐⭐⭐⭐
54. Module registry	Public (namespace/module/provider), private (TFC), governance	⭐⭐⭐

🔑 Category 7 — Critical Rules

```
Module interface rules:
  variables.tf = what callers MUST or CAN provide
  outputs.tf   = what callers GET BACK (public API)
  Never expose internal implementation details as outputs
  Never make callers configure what should be an organizational default

Module versioning rules:
  Reusable module providers: ">= 4.0, < 6.0" (permissive)
  Root module providers:     "~> 5.1"          (tight)
  Git sources:  always ?ref=v2.1.0 (tag, never branch)
  Registry:     always version = "= 5.1.4" (exact in prod)
  .terraform.lock.hcl locks providers NOT modules

moved block rules:
  Requires Terraform 1.1+
  Keep until ALL environments have applied
  Remove in separate PR after all envs updated
  Preferred over terraform state mv for planned refactoring

terraform import rules:
  Write config first, import, then plan until 0 changes
  TF 1.5+: use import blocks + -generate-config-out (saves hours)
  Import ID format varies by resource — always check provider docs
  After import: terraform state show to see actual attributes
```

🎯 Category 7 — Top 5 Interview Questions to Master

1. **"Explain the anatomy of a Terraform module — what files does it need and why?"** — variables.tf (interface), outputs.tf (API), versions.tf (permissive), README.md
2. **"What's the difference between a root module, child module, and wrapper module?"** — root=orchestrator, child=focused reusable, wrapper=standards enforcer with narrow interface
3. **"I renamed an EC2 resource from `aws_instance.web` to `aws_instance.web_server`. What will Terraform do and how do I prevent the destroy/recreate?"** — `moved` block, from/to syntax, 0 changes

4. **"Walk me through importing an existing RDS instance into Terraform management"** — write config, import command with DB identifier, state show, update config, iterate until 0 changes, TF 1.5+ generate-config-out
5. **"Should I pin module versions with `= 5.1.4` or `~> 5.0`?"** — depends on context: exact in root modules for reproducibility, permissive in reusable modules so callers aren't blocked by provider version conflicts

---

> **Next:** Category 8 — Security & Secret Management (Topics 55–61) Type `Category 8` to continue, `quiz me` to be tested on Category 7, or `deeper` on any specific topic.